UNIVERSIDAD DE CARABOBO

FACULTAD DE CIENCIAS Y TECNOLOGIA

FUNDAMENTOS DE PROGRAMACIÓN

Proyecto
Ferrys Margarita

Participantes:

Rances Colotto 30.997.818

Ismael Armada 31.390.408

Sección 8

Valencia, 3 Abril de 2026

Explicación general del enunciado y objetivos

El problema consiste en simular la operación diaria de una empresa de ferrys que conecta la isla de Margarita con el continente. La empresa dispone de tres ferrys con diferentes capacidades y características (uno de tipo Express y dos de tipo Tradicional). Cada vehículo que llega al puerto tiene asociado un tipo (liviano, rústico, microbús, carga, ambulancia, bomberos o policial), cantidad de pasajeros (adultos y tercera edad), clase de pasaje (VIP/Ejecutiva o 1ra/Turista), peso, hora de llegada, placa y una preferencia de ferry (Express o Tradicional). Los vehículos de emergencia (ambulancia, bomberos, policial) tienen prioridad absoluta.

El objetivo es **simular minuto a minuto** la llegada de vehículos, su asignación a colas de espera, el proceso de carga en los ferrys (respetando prioridades y restricciones de capacidad), el zarpe de los barcos bajo condiciones específicas, y finalmente generar un reporte detallado de cada viaje y estadísticas globales del día.

Contexto y propósito del software desarrollado

El software fue desarrollado en lenguaje C estándar, sin librerías externas, para ejecutarse en un entorno académico. Su propósito es **modelar el comportamiento real de un sistema logístico-portuario** con restricciones de capacidad (vehículos, pasajeros por clase, peso), reglas de prioridad (emergencias primero) y condiciones de despacho (mínimo de vehículos, cola vacía o imposibilidad de cargar más). Permite a los operadores evaluar la eficiencia de la flota, identificar cuellos de botella (máxima cola de espera) y calcular ingresos y pasajeros no trasladados.

Análisis de la solución

Cómo se organizan y almacenan los datos

Los datos se organizan en tres niveles:

1. **Entrada (**proy1.in**)**:
    a. Primera línea: orden inicial de los tres ferrys (IDs 1,2,3).
    b. Líneas siguientes: cada vehículo con 9 campos (código, pasajeros, clase, peso, hora, placa, ferry destino).

   c. Se almacenan en un arreglo Veh lista[MAX_VEH] en el orden de llegada (garantizado por el enunciado).
2. **Estructuras en memoria**:
   a. Veh: representa cada vehículo con todos sus atributos.
   b. Ferry: estado fijo (capacidades) y dinámico (vehículos a bordo, contadores, estado, tiempo restante).
   c. Cola: cola circular de índices a lista[] para manejar espera de vehículos y disponibilidad de ferrys.
   d. Stats: acumuladores globales del día.
3. **Salida (**proy1.out **y pantalla)**:
   a. Reporte de cada viaje (número, ferry, cantidad de vehículos, pasajeros, peso, ingreso, lista de placas).
   b. Estadísticas finales (totales, vehículo más frecuente, máximo de cola, pasajeros perdidos).

Justificación de las decisiones tomadas

Decisión	Justificación
Usar **cola circular** para espera de vehículos	Permite inserción al frente (cuando un vehículo normal es bajado por una emergencia) sin desplazar memoria, en O(1).
Índices en colas en lugar de copiar vehículos	Evita duplicar estructuras grandes y mantiene coherencia con los datos originales.
Simulación por minutos (bucle de 0 a 1440)	Se ajusta al requerimiento de "minuto a minuto" y permite manejar llegadas exactas y tiempos de carga/zarpe.
Prioridad estricta (emergencias → cola normal del ferry)	Cumple la especificación: las emergencias abordan inmediatamente, bajando vehículos normales si es necesario.
Condición de zarpe con **verificación de "ningún vehículo en cola puede subir"**	Evita que el ferry zarpe prematuramente si aún hay espacio pero los vehículos en espera no caben por peso, pasajeros o asientos.
Dos destinos de impresión (stdout y archivo)	Cumple con mostrar en pantalla y guardar en proy1.out simultáneamente.

Definición de los structs utilizados

Definición de los structs utilizados

```
typedef struct {
    int  cod, npa, npt, tpa, tpt, peso, hora;
    char placa[16];
    int  tipo_ferry;
} Veh;
```

Veh – Almacena toda la información de un vehículo según el archivo de entrada.

```
typedef struct {
    int buf[MAX_VEH], head, tail, size;
} Cola;
```

Cola – Cola circular de enteros (índices a Veh lista[]).

```
typedef struct {
    char      nombre[32];
    TipoFerry tipo;
    int       id;
    int       cap_veh, min_veh, cap_pas, cap_vip, cap_eje;
    float     cap_ton;
    int       t_ciclo;
    int       abordo[MAX_VEH];
    int       n_veh, pas, pas_3ra, vip_us, eje_us;
    float     ton_car;
    Estado    estado;
    int       t_rest, n_viajes;
    float     ingresos;
} Ferry;
```

Ferry – Capacidades fijas + estado dinámico de carga/viaje.

```
typedef struct {
    int   tot_veh, tot_pas, pas_perd;
    float tot_ing;
    int   frec[8];
    int   max_cola, h_cola;
} Stats;
```

Stats – Estadísticas globales del día.

Arreglos u otras estructuras aplicadas

Estructura	Tipo	Uso
Veh lista[MAX_VEH]	Arreglo estático	Almacena todos los vehículos del día
Ferry flota[N_FERRYS]	Arreglo estático	Los 3 ferrys
Cola q_exp, q_trad, q_emerg, q_muelle	Variables locales	Colas de espera y disponibilidad
int abordo[MAX_VEH] dentro de Ferry	Arreglo dinámico por viaje	Índices de vehículos cargados actualmente
int frec[8] dentro de Stats	Arreglo de frecuencias	Conteo por tipo de vehículo (índices 1..7)

Diagrama de relaciones

```
[Archivo proy1.in]
       │
       ▼
  lista[MAX_VEH] (Veh)
       │
       ├──► colaExpress (índices)
       ├──► colaTradicional (índices)
       └──► colaEmergencias (índices)

[Simulación minuto a minuto]
       │
       ├──► Ferry[0] (Express)
       ├──► Ferry[1] (Tradicional)
       └──► Ferry[2] (Tradicional)

       │
       ▼
[Reportes: proy1.out + pantalla]
```

Algoritmos implementados

Algoritmo principal de simulación

```
[1] encolar vehículos que llegan en min_act
        emergencias → q_emerg
        tipo_ferry=0 → q_exp
        tipo_ferry=1 → q_trad

  [2] actualizar estadística de espera máxima

  [3] si hay ferry en muelle y t_libre ≤ min_act:
        prioridad 1: embarcar emergencia (bajando normales si es necesario)
        prioridad 2: embarcar frente de la cola del ferry

        si puede_zarpar:
            acumular estadísticas del viaje
            imprimir reporte del viaje
            reiniciar contadores del ferry
            poner ferry en EN_VIAJE con t_rest = t_ciclo
            sacar siguiente ferry de q_muelle (si existe)

  [4] decrementar t_rest de los ferrys en viaje
        si t_rest ≤ 0 → pasar a EN_ESPERA, encolar en q_muelle

  [5] condición de fin:
        si no hay más vehículos por llegar
        y las tres colas de espera están vacías
        y el ferry en muelle no alcanza el mínimo
          → terminar

  min_act++
```

Pruebas y comportamiento observado

Comportamiento del código paso a paso

1. **Lectura de datos**
    a. Se abre el archivo proy1.in.
    b. Se lee el orden de atención de los ferries (por ejemplo, 1 3 2).
    c. Se cargan los vehículos en un arreglo con sus atributos (código, pasajeros, peso, placa, tipo de ferry destino, etc.).
2. **Inicialización de la flota**
    a. Se definen tres ferries:
        i. *Lilia Concepción* (Express)
        ii. *La Isabela* (Tradicional)
        iii. *La Margaritena* (Tradicional)
    b. Cada uno tiene capacidades fijas y estado dinámico (vehículos a bordo, pasajeros, peso cargado, etc.).
3. **Simulación minuto a minuto**
    a. Se avanza en el tiempo desde la llegada del primer vehículo hasta 24 horas después.
    b. En cada minuto:
        i. Se encolan los vehículos que llegan en ese instante.
        ii. Si hay un ferry en el muelle y está en hora de carga, se intenta embarcar vehículos con prioridad:
            1. Emergencias (sacan al último vehículo normal si es necesario).
            2. Vehículos normales de su cola correspondiente.
        iii. Se verifica si el ferry puede zarpar (según reglas del enunciado).
        iv. Se registran estadísticas de espera (longitud máxima de cola y hora).
        v. Se actualizan los ferries en viaje (descuentan tiempo restante).
    c. Finaliza cuando no quedan vehículos por procesar y las colas están vacías.
4. **Generación de reportes**
    a. Por cada viaje completado: ferry, número de viaje, cantidad de vehículos, pasajeros, peso, ingreso y lista de placas.
    b. Estadísticas finales:
        i. Total de vehículos y pasajeros transportados.
        ii. Total de ingresos.
        iii. Pasajeros no trasladados.
        iv. Tipo de vehículo más frecuente.
        v. Mayor cantidad de vehículos en espera y hora en que ocurrió.

5. **Salida**
    a. Los resultados se muestran en pantalla y se guardan en proy1.out.

Pruebas realizadas y comportamiento observado

1. Identificación y manejo de vehículos repetidos en la cola

Capacidad de identificar automáticamente si algún carro de la cola es repetido (ya que automáticamente lo ignora), permitiendo evitar errores como generar **Comportamiento:**
El programa no realiza una verificación explícita de duplicados por placa o código. Sin embargo, al leer el archivo de entrada, procesa cada línea como un vehículo distinto y lo encola según su hora de llegada. Si el archivo contiene registros duplicados, el programa los tratará como vehículos diferentes, lo que podría llevar a inconsistencias (por ejemplo, el mismo vehículo apareciendo dos veces en la simulación).

Validación de existencia y formato del archivo de entrada

Si el programa no identifica ningún dato de `proy1.in`*, automáticamente el programa no iniciará advirtiendo al usuario, así como también si no detecta ninguno de los datos esperados en dicho archivo.*

Comportamiento:

- Si el archivo `proy1.in` no existe o no puede abrirse, la función `leer_datos()` retorna `-1` y `main()` muestra el mensaje:
  `"Error: no se pudo abrir proy1.in"`
- Si el archivo está vacío o no contiene los datos esperados (por ejemplo, falta la primera línea con el orden de ferries o no hay vehículos con el formato correcto), el programa puede leer 0 vehículos. En ese caso, la simulación se ejecutaría sin vehículos, generando reportes sin actividad, pero **no se advierte explícitamente** que no se encontraron vehículos válidos.

Reflexión y concluciones

Durante la ejecución del proyecto, se identificaron diversos desafíos y estrategias de resolución. Lo más complicado fue la gestión de la procrastinación. No obstante, se logró facilitar de manera significativa el desarrollo del proyecto, aprovechando colectivamente las extensiones de Visual Studio Code (VSCode) como:

- efactorización de código.
- Documentación continua del mismo.
- Depuración (debugging) eficiente.

Adicionalmente, aprender a utilizar Git para el control de versiones. Esto permitió mantener de forma constante un **mínimo entregable** del proyecto en todo momento, asegurando así la funcionalidad básica y la trazabilidad de los avances

En cuanto al trabajo con el código, las principales dificultades enfrentadas fueron:

- La parametrización de las funciones.
- La implementación de la lógica de la simulación.

Para poder completar las tareas dentro del plazo establecido, recurrimos constantemente a Stack Overflow como fuente de consulta y soporte técnico, lo que permitió resolver los inconvenientes de manera ágil y eficiente.

Cosas a mejorar

No se advierte cuando no hay vehículos válidos en el archivo

Si el archivo proy1.in existe pero no contiene ningún vehículo con el formato correcto (por ejemplo, líneas mal escritas o solo comentarios), leer_datos() retorna con n = 0. El programa continúa ejecutando simular() sin vehículos, generando reportes vacíos y sin advertir al usuario.

Solución:
Validar después de la lectura que se haya cargado al menos un vehículo. En caso contrario, mostrar un mensaje de error y salir.

La condición de zarpe no considera vehículos de emergencia en cola

puede_zarpar() verifica si algún vehículo normal puede subir, pero no tiene en cuenta que podría haber emergencias esperando. Según la lógica de prioridad, si hay emergencias, el ferry no debería zarpar hasta atenderlas.

Solución:
Agregar una verificación de emergencias pendientes antes de permitir el zarpe.